

# Order-six torsion in an ordered graph configuration space

## Abstract

We construct an element of exact order six in the second integral homology of the ordered configuration space of four points on a finite graph. This disproves the torsion-freeness conjecture stated by Chettih and Lütgehetmann (Algebr. Geom. Topol., 2018). Twice the constructed class has order three, giving an affirmative answer, in the ordered setting, to the question of odd torsion highlighted by Mamun, Nalikka, and Ramos (arXiv, 2026). The graph has six vertices and thirteen edges, or eight vertices and fifteen edges in a homeomorphic simple presentation. The proof uses an integral cycle  $z$ , an integral chain with boundary  $6z$ , and a cocycle modulo six evaluating to one on  $z$ . These identities establish the exact order without a full homology computation. We construct the certificate by unit cancellation over  $\mathbb{Z}[S_4]$  and then over  $\mathbb{Z}$ , followed by Bézout identities and explicit lifting to the original cubical complex. A complete program in the appendix generates the certificate from the graph and checks all identities using exact arithmetic, including the cocycle condition on every labelled three-dimensional cube.

**Note.** This paper was generated entirely by AI, including MiMo, using an automated research pipeline developed by Chenghua Liu and Hanyu Li.

## 1 Introduction

The ordered configuration space

$$F_n(G) = \{(x_1, \dots, x_n) \in G^n : x_i \neq x_j \text{ whenever } i \neq j\}$$

records the positions of  $n$  distinguishable particles on a graph  $G$ . Although a graph has free abelian integral homology, the requirement that particles remain distinct introduces relations not present in the underlying one-dimensional space. The question is whether these relations can create torsion: can a nonbounding integral cycle have a nonzero integer multiple that bounds? Throughout, a finite graph is identified with its topological realization; parallel edges are allowed.

The distinction between ordered and unordered configurations is essential. Write  $U_n(G) = F_n(G)/S_n$ . For a finite connected graph and  $n \geq 2$ , Ko and Park describe  $H_1(U_n(G); \mathbb{Z})$  in terms of graph invariants: its torsion consists of copies of  $\mathbb{Z}/2$  indexed by nonplanar triconnected components [5, Theorem 3.16]. By contrast, their calculation for two ordered particles gives torsion-free integral homology in every degree [5, Theorem 3.25 and the paragraph following it]. Thus torsion in the unordered quotient does not by itself answer the ordered question.

Chettih and Lütgehetmann proved that  $H_q(F_n(G); \mathbb{Z})$  is torsion-free for every  $n$  and  $q$  when  $G$  is a tree with loops, meaning an iterated wedge of stars and circles. They also construct geometric generators for these groups, and conjecture torsion-freeness for all finite graphs [2, Theorem A and Conjecture 3]. An earlier assertion of the general result was withdrawn because the chosen bases did not give the required splitting of a Mayer–Vietoris spectral sequence [2, p. 2446]. This leaves an integral issue: free abelian groups on the pages of a spectral sequence do not preclude torsion in quotients by images of differentials.

In their 2025 preprint, Hainaut, Knudsen, and Wawrykow report that no torsion example was known for ordered graph configuration spaces and restate torsion-freeness as a folklore conjecture

[3, Section 1.1 and Conjecture 1.7]. Their Theorem 1.2 gives leading Betti-number asymptotics independent of the characteristic of the coefficient field, under its graph and degree hypotheses. The resulting asymptotic restriction on possible torsion [3, Corollary 1.8] does not exclude its occurrence at a fixed particle number. Computations of rational homology representations for several graph families [8] likewise leave integral torsion undetermined.

In their September 2026 preprint, Mamun, Nalikka, and Ramos [6, Section 4.3.1] again describe torsion-freeness for ordered configurations as an open conjecture and single out the existence of odd torsion as a central question in graph-configuration homology. Their results on anchored configuration spaces of cographs, where collisions are permitted, bound torsion exponents for fixed homological degree, particle number, and anchor count [6, Remark 4.14 and Corollary 4.18], but do not exhibit torsion in the ordinary ordered space.

We disprove the ordered torsion-freeness conjecture and answer the odd-torsion existence question affirmatively in the ordered setting. Both conclusions follow from a single degree-two class of exact order six for four labelled particles. Define  $G$  by starting with  $K_4$  on vertices  $x_1, x_2, y_1, y_2$ , adding vertices  $a, b$  and edges  $ax_1, ax_2, by_1, by_2$ , and joining  $a$  to  $b$  by three distinct parallel edges. This graph has six vertices and thirteen edges; see Figure 1.

**Theorem 1.** *For the graph  $G$  defined above,  $H_2(F_4(G); \mathbb{Z})$  contains a class  $[z]$  of exact order six. Consequently,  $2[z]$  has order three and  $3[z]$  has order two.*

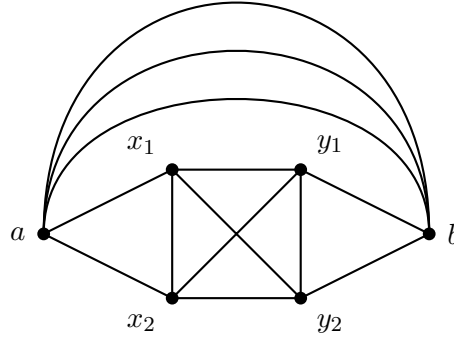


Figure 1: The graph  $G$ . The crossing inside the middle square is not a vertex.

Subdividing two of the parallel edges once gives a homeomorphic simple graph: replace the three  $ab$  edges by  $ab, au, ub, av, vb$ , where  $u, v$  are new vertices. Thus the theorem also holds for a simple graph with eight vertices and fifteen edges. The graph  $G$  contains  $K_{3,3}$  with bipartition  $\{a, y_1, y_2\}$  and  $\{b, x_1, x_2\}$ , using one  $ab$  edge, and hence lies outside the tree-with-loops class.

The proof is a finite, exact calculation in the cubical model of Abrams. Rather than compute a complete integral homology decomposition, we construct three objects: a cycle  $z$ , a chain bounding  $6z$ , and a modulo-six cocycle evaluating to one on  $z$ . The bounding chain proves that the order divides six; the cocycle proves the reverse divisibility. We use the established cubical model and elementary unit cancellation to obtain this explicit certificate. Encoding chains over  $\mathbb{Z}[S_4]$  reduces the size of the calculation without taking the quotient by particle permutations. Section 2 describes the certificate and its topological interpretation. Appendix A explains the construction and gives the complete program, including a verifier that works in the unreduced complex.

## 2 The cubical certificate

Number the core vertices by

$$a = 0, \quad b = 1, \quad x_1 = 2, \quad x_2 = 3, \quad y_1 = 4, \quad y_2 = 5.$$

The following ordered list fixes the orientations and indices of the thirteen core edges, starting with index zero:

$$\begin{aligned} &(0, 1), (0, 1), (0, 1), (0, 2), (0, 3), (1, 4), (1, 5), \\ &(2, 3), (2, 4), (2, 5), (3, 4), (3, 5), (4, 5). \end{aligned} \tag{1}$$

For the core edge of index  $e$  with endpoints  $(u, v)$ , use the oriented path

$$u \longrightarrow 6 + 2e \longrightarrow 7 + 2e \longrightarrow v,$$

and number its three edges  $3e, 3e + 1, 3e + 2$ . The resulting graph  $\Gamma$  has 32 vertices and 39 edges.

Let  $D_4(\Gamma)$  be the cubical subcomplex of  $\Gamma^4$  consisting of products of four graph cells with pairwise disjoint closures. The ordered stable-equivalence theorem of Prue and Scrimshaw [7, Theorem 3.2], refining Abrams' discretization theorem [1], applies to a connected graph with at least four vertices when every path between distinct vertices of degree other than two has at least three edges and every homotopically nontrivial closed path has at least five edges. In  $\Gamma$  the shortest such vertex-to-vertex path has three edges, and the girth is six. Therefore the inclusion  $D_4(\Gamma) \hookrightarrow F_4(\Gamma)$  is a homotopy equivalence. Subdivision preserves the underlying topological graph, so

$$D_4(\Gamma) \simeq F_4(\Gamma) \cong F_4(G). \tag{2}$$

Write  $C_q = C_q(D_4(\Gamma); \mathbb{Z})$  and  $\partial_q : C_q \rightarrow C_{q-1}$  for the cellular boundary map.

We orient each cube by increasing indices of its moving graph edges, rather than by increasing particle labels. For a  $q$ -dimensional cube  $c$ , its boundary is

$$\partial c = \sum_{j=1}^q (-1)^{j-1} (c_{j,+} - c_{j,-}), \tag{3}$$

where  $c_{j,+}$  and  $c_{j,-}$  replace the  $j$ th moving edge by its terminal and initial vertices, respectively. Labels are retained on every face. This choice of orientations makes relabelling act without an additional orientation sign.

For the finite encoding, tokens  $0, \dots, 31$  denote vertices of  $\Gamma$ , and token  $32 + e$  denotes open edge  $e$ . A *canonical cube* is an increasing four-tuple of tokens with pairwise disjoint closures. For a permutation  $p = (p(0), \dots, p(3))$  of  $\{0, 1, 2, 3\}$ , write  $[p; c]$  for the labelled cube assigning token  $c_i$  to particle  $p(i) + 1$ . To form a face, replace an edge token by an endpoint and sort the resulting tokens. If the sorted order lists old coordinate indices as  $q(0), \dots, q(3)$ , the new particle assignment is  $p \circ q$ . The surviving moving edges retain their relative order, so the sign is exactly that in (3).

The construction in Appendix A produces an integral 2-chain  $z$ , an integral 3-chain  $b$ , and a homomorphism  $\phi : C_2 \rightarrow \mathbb{Z}/6$ . Its input is only the graph in (1); lexicographic cell and permutation orders and deterministic pivot rules specify every coefficient. The resulting cycle has the form

$$z = \sum_{(c, \epsilon_c) \in \mathcal{L}} \epsilon_c \sum_{p \in S_4} \text{sgn}(p) [p; c], \tag{4}$$

where the program constructs  $\mathcal{L}$ , consisting of 409 canonical 2-cubes with coefficients  $\epsilon_c \in \{1, -1\}$ , and verifies this alternating form. Thus  $z$  has 9,816 nonzero labelled coefficients, all  $\pm 1$ . The

supports of  $b$  and  $\phi$  have sizes 236,992 and 214,457, respectively. Coefficients absent from a sparse representation are zero.

The identities verified by the program are

$$\partial_2 z = 0, \quad \partial_3 b = 6z, \quad \phi \partial_3 = 0, \quad \phi(z) = 1 \in \mathbb{Z}/6. \quad (5)$$

The first two are integral identities; the last two are in  $\mathbb{Z}/6$ . The function `check` constructs the boundaries of  $z$  and  $b$  directly from (3). To verify the cocycle condition, it enumerates each set of three mutually disjoint graph edges, each stationary vertex outside their endpoints, and all 24 assignments of particle labels. These are precisely all the three-dimensional cubes of  $D_4(\Gamma)$ : each such cube has three moving particles and one stationary particle. There are

$$174,772 \cdot 24 = 4,194,528$$

labelled cubes to check. In particular, the cocycle test is not restricted to cubes in the support of  $b$ . Every equality is tested using exact integer arithmetic or reduction modulo six.

*Proof of Theorem 1.* The first identity in (5) defines a class  $[z] \in H_2(D_4(\Gamma); \mathbb{Z})$ , and the second gives  $6[z] = 0$ . The third implies that evaluation by  $\phi$  descends to a homomorphism

$$H_2(D_4(\Gamma); \mathbb{Z}) \longrightarrow \mathbb{Z}/6.$$

By the fourth identity, this homomorphism sends  $[z]$  to 1. Consequently,  $k[z] = 0$  implies  $k = 0$  in  $\mathbb{Z}/6$ , or equivalently  $6 \mid k$ . Thus  $[z]$  has exact order six. The equivalence (2) transports this class to  $F_4(G)$ .  $\square$

The certificate identifies a failure of saturation:

$$6z \in \text{im } \partial_3, \quad z \in \ker \partial_2 \setminus \text{im } \partial_3.$$

In other words, the boundary subgroup contains a nonzero multiple of a cycle but not the cycle itself. The modulo-six cocycle detects precisely the obstruction that disappears after passing to rational coefficients. This is an integral phenomenon in the ordered space, not torsion introduced by an unordered quotient.

## References

- [1] A. D. Abrams. *Configuration Spaces and Braid Groups of Graphs*. Ph.D. thesis, University of California, Berkeley, 2000.
- [2] S. Chettih and D. Lütgehetmann. The homology of configuration spaces of trees with loops. *Algebr. Geom. Topol.* **18** (2018), no. 4, 2443–2469. <https://doi.org/10.2140/agt.2018.18.2443>.
- [3] L. Hainaut, B. Knudsen, and N. Wawrykow. *Representation asymptotics in the homology of pure graph braid groups*. arXiv:2510.00201, 2025. <https://arxiv.org/abs/2510.00201>.
- [4] M. Jöllenbeck and V. Welker. *Resolution of the residue class field via algebraic discrete Morse theory*. arXiv:math/0501179, 2005. <https://arxiv.org/abs/math/0501179>.
- [5] K. H. Ko and H. W. Park. Characteristics of graph braid groups. *Discrete Comput. Geom.* **48** (2012), no. 4, 915–963. <https://doi.org/10.1007/s00454-012-9459-8>.
- [6] A. Mamun, J. Nalikka, and E. Ramos. *Universality in the algebra and topology of cographs*. arXiv:2609.04554, September 2026. <https://arxiv.org/abs/2609.04554>.
- [7] P. Prue and T. Scrimshaw. Abrams’s stable equivalence for graph braid groups. *Topology Appl.* **178** (2014), 136–145. <https://doi.org/10.1016/j.topol.2014.09.009>.
- [8] E. Ramos and C. H. Yun. *Computing stable homology representations of graph configuration spaces*. arXiv:2606.13813, 2026. <https://arxiv.org/abs/2606.13813>.

## A Complete construction and verification

The program below is both a finite specification of the certificate and an exact verifier of (5). We first explain why its reductions and lifting formulas produce chains and cochains in the original cubical coordinates. Only  $\partial_3$  is needed for the construction; the verification computes  $\partial_2 z$  separately.

Let  $R = \mathbb{Z}[S_4]$ . With the orientation convention above, the labelled chain groups are free left  $R$ -modules on the canonical cubes. Write  $f_j$  for the canonical basis in degree three and  $e_i$  for that in degree two, so

$$\partial_3 f_j = \sum_i M_{ij} e_i.$$

Thus a source coefficient  $x_j \in R$  contributes  $x_j M_{ij}$  to row  $i$ . The same notation is used for the bases at subsequent reduction steps. The matrix is stored by its nonzero rows and columns, and each entry is a finite integer combination of permutations. It is not augmented to  $\mathbb{Z}$ : all 24 permutation coefficients are retained.

Suppose  $a = M_{rc} = \pm g$  for  $g \in S_4$ . For each  $j \neq c$ , replace  $f_j$  by

$$f'_j = f_j - M_{rj} a^{-1} f_c.$$

Its boundary has zero  $e_r$ -coefficient. Deleting row  $r$  and column  $c$  therefore leaves the matrix

$$M'_{ij} = M_{ij} - M_{rj} a^{-1} M_{ic} \quad (i \neq r, j \neq c). \quad (6)$$

This formula follows by taking the boundary of  $f'_j$ ; the order of its factors cannot be changed in the noncommutative ring  $R$ .

The remaining target basis vectors  $e_i$  have not changed. Hence a target chain for the reduced matrix is embedded in the previous target module by giving it zero coefficient at  $e_r$ . A reduced source chain with coefficients  $x_j$  is lifted by restoring the coefficient

$$x_c = -\left(\sum_{j \neq c} x_j M_{rj}\right) a^{-1}. \quad (7)$$

Indeed, its row  $r$  vanishes, and its other boundary coefficients are those obtained by applying  $M'$  to the reduced source. In particular, the target chain  $z$  requires only zero extension, not a further change of coordinates.

To lift a homomorphism from the reduced target module to  $\mathbb{Z}/6$  that annihilates the image of the reduced map, extend it on the eliminated row by

$$\phi(ue_r) = -\sum_{i \neq r} \phi(ua^{-1} M_{ic} e_i) \quad (u \in S_4). \quad (8)$$

Here  $\phi$  is additive, not assumed  $R$ -linear. The formula defines it on an integral basis of  $Re_r$ . It makes  $\phi$  vanish on all multiples of  $\partial_3 f_c$ , since the  $e_r$ -coefficient of  $ua^{-1} \partial_3 f_c$  is  $u$ . It also annihilates all other original columns and their  $R$ -multiples: their differences from the reduced columns are multiples of the pivot column. Its values on the embedded target chain are unchanged. These observations justify reversing the recorded cancellations to lift both the bounding chain and the detecting cochain.

For integer pivots the same argument applies with  $a = \pm 1$ . Cancellation of invertible entries is the elementary operation behind algebraic discrete Morse theory; see [4, Section 2] for that framework. The noncommutative formulas needed here have been derived explicitly, rather than inferred from a central-coefficient version of the theory.

Canonical cubes and permutations are ordered lexicographically. At each step the program considers the shortest row and the shortest column containing an entry of the form  $\pm g$  (or  $\pm 1$  in

the integer matrix), breaking ties by index. Within each candidate vector it chooses a pivot incident to a shortest opposite vector, again breaking ties by index. Of these two candidates it takes the one with smaller  $(\ell_r - 1)(\ell_c - 1)$ , where  $\ell_r, \ell_c$  are the lengths of the pivot row and column, preferring the row candidate on a tie. A step removes one row and one column, so the reduction terminates.

For this graph, 131,107 group-ring pivots leave a nonzero matrix with 3,361 rows and 61 columns over  $R$ . Rows and columns that are zero are omitted at this stage; target chains and functionals are assigned zero on omitted rows. Expanding in the permutation basis gives an integer matrix with 80,664 rows and 1,464 columns. A further 118 integer unit pivots leave nonzero columns of the form

$$d_j z_0, \quad z_0 \text{ primitive}, \quad \gcd_j d_j = 6.$$

These assertions are checked directly during construction: the first nonzero column, divided by the greatest common divisor of its entries, defines  $z_0$ , and every other column is tested for equality with its claimed integer multiple of  $z_0$ .

The function `bezout_vector` applies the extended Euclidean algorithm successively. Applied to the entries of  $z_0$ , it produces an integer functional  $\lambda$  satisfying  $\lambda(z_0) = 1$ ; applied to the integers  $d_j$ , it produces coefficients  $s_j$  with  $\sum_j s_j d_j = 6$ . Thus the residual source chain  $\sum_j s_j f_j$  has boundary  $6z_0$ , and  $\lambda$  modulo six annihilates every residual column while taking value one on  $z_0$ . Formulas (7) and (8), first over  $\mathbb{Z}$  and then over  $R$ , recover  $b$  and  $\phi$  in the original coordinates; zero extension recovers  $z$ . The calculation then returns to the original graph cells and verifies all four identities (5), without using the reduced matrices or the recorded pivots in the verifier.

The listing is one complete program. It requires Python 3.9 or later and only the standard library, with no input files, random choices, downloaded data, or supplementary modules. All coefficient operations use arbitrary-precision integers; cochain values are reduced modulo six. Run it with assertions enabled, for example as `python3 certificate.py`; the program rejects execution with `-O`. Successful termination prints `"verified": true`. An assertion failure does not certify the theorem.

The complete listing was executed with CPython 3.13.5 on 64-bit Linux. Its exact output confirms 9,816 nonzero coefficients of  $z$ , 236,992 of  $b$ , 214,457 nonzero values of  $\phi$ , and 4,194,528 labelled 3-cubes checked, with  $\phi(z) = 1$  modulo six. The construction used approximately 1.2 GB of peak resident memory in that execution; this resource measurement is not part of the certificate.

```
# Complete construction and verification; Python 3.9 or later.
# No input files or third-party modules are used.
import functools
import heapq
import itertools
import json
import math
from collections import deque
from types import SimpleNamespace

class SymmetricGroup:

    def __init__(self, n):
        self.n = n
        self.permutations = list(itertools.permutations(range(n)))
        self.index = {p: i for i, p in enumerate(self.permutations)}
        self.identity = self.index[tuple(range(n))]
        self.inverses = []
        for p in self.permutations:
            inverse = [0] * n
            for i, j in enumerate(p):
                inverse[j] = i
```

```

        self.inverses.append(self.index[tuple(inverse)])
    self.multiply = functools.lru_cache(maxsize=1000000)(self._multiply)

    def _multiply(self, a, b):
        p, q = (self.permutations[a], self.permutations[b])
        return self.index[tuple((p[i] for i in q))]

    def add_product(self, destination, left, right, sign=1):
        for g, a in left.items():
            for h, b in right.items():
                k = self.multiply(g, h)
                new = destination.get(k, 0) + sign * a * b
                if new:
                    destination[k] = new
                else:
                    destination.pop(k, None)

def is_unit(value):
    return len(value) == 1 and abs(next(iter(value.values()))) == 1

class GroupRingMatrix:

    def __init__(self, group, nrows, columns):
        self.group = group
        self.rows = [{ } for _ in range(nrows)]
        self.cols = columns
        for j, col in enumerate(columns):
            for i, a in col.items():
                assert a
                self.rows[i][j] = a

    def unit_reduce(self, on_pivot=None):
        rv = [0] * len(self.rows)
        cv = [0] * len(self.cols)
        rh = [(len(row), i, 0) for i, row in enumerate(self.rows) if row]
        ch = [(len(col), j, 0) for j, col in enumerate(self.cols) if col]
        heapq.heapify(rh)
        heapq.heapify(ch)
        count = 0

    def candidate(heap, vectors, versions, opposite):
        while heap:
            _, index, version = heap[0]
            if version != versions[index] or not vectors[index]:
                heapq.heappop(heap)
                continue
            units = [k for k, a in vectors[index].items() if is_unit(a)]
            if not units:
                heapq.heappop(heap)
                continue
            k = min(units, key=lambda t: (len(opposite[t]), t))
            cost = (len(vectors[index]) - 1) * (len(opposite[k]) - 1)
            return (cost, index, k)
        return None

    while True:
        rc = candidate(rh, self.rows, rv, self.cols)
        cc = candidate(ch, self.cols, cv, self.rows)
        if rc is None and cc is None:
            break
        if cc is None or (rc is not None and rc[0] <= cc[0]):

```

```

        _, r, c = rc
    else:
        _, c, r = cc
    unit = self.rows[r][c]
    g, sign = next(iter(unit.items()))
    assert abs(sign) == 1
    inverse = self.group.inverses[g]
    prow = list(self.rows[r].items())
    pcol = list(self.cols[c].items())
    if on_pivot:
        on_pivot(r, c, unit)
    for j, b in prow:
        if j == c:
            continue
        factor = {
            self.group.multiply(h, inverse): sign * coefficient
            for h, coefficient in b.items()
        }
        col = self.cols[j]
        del col[r]
        for i, x in pcol:
            if i == r:
                continue
            new = col.get(i, {}).copy()
            self.group.add_product(new, factor, x, sign=-1)
            if new:
                col[i] = new
                self.rows[i][j] = new
            else:
                col.pop(i, None)
                self.rows[i].pop(j, None)
    for i, _ in pcol:
        self.rows[i].pop(c, None)
    self.rows[r] = {}
    self.cols[c] = {}
    for i, _ in pcol:
        rv[i] += 1
        if self.rows[i]:
            heapq.heappush(rh, (len(self.rows[i]), i, rv[i]))
    for j, _ in prow:
        cv[j] += 1
        if self.cols[j]:
            heapq.heappush(ch, (len(self.cols[j]), j, cv[j]))
    count += 1
    if len(rh) > max(10000, 4 * len(self.rows)):
        rh = [(len(row), i, rv[i]) for i, row in enumerate(self.rows) if row]
        heapq.heapify(rh)
    if len(ch) > max(10000, 4 * len(self.cols)):
        ch = [(len(col), j, cv[j]) for j, col in enumerate(self.cols) if col]
        heapq.heapify(ch)
    return count

```

```

class SparseMatrix:

```

```

    def __init__(self, nrow: int, columns: list[dict[int, int]]):
        self.rows = [{ } for _ in range(nrow)]
        self.cols = columns
        for j, col in enumerate(columns):
            for i, a in col.items():
                if not a:
                    raise ValueError("Explicit zero entry")
                self.rows[i][j] = a

```

```

def unit_reduce(self, on_pivot=None):
    """Schur-complement cancellation at unit pivots; exact over Z.

    At each step choose the less expensive of a shortest-row and
    shortest-column candidate. Stale heap entries are lazily discarded.
    """
    rversions = [0] * len(self.rows)
    cversions = [0] * len(self.cols)
    rh = [(len(row), i, 0) for i, row in enumerate(self.rows) if row]
    ch = [(len(col), j, 0) for j, col in enumerate(self.cols) if col]
    heapq.heapify(rh)
    heapq.heapify(ch)
    rank = 0

    def candidate(heap, vectors, versions, opposite):
        while heap:
            _, idx, version = heap[0]
            if version != versions[idx] or not vectors[idx]:
                heapq.heappop(heap)
                continue
            unit = [k for k, a in vectors[idx].items() if abs(a) == 1]
            if not unit:
                heapq.heappop(heap)
                continue
            k = min(unit, key=lambda x: (len(opposite[x]), x))
            cost = (len(vectors[idx]) - 1) * (len(opposite[k]) - 1)
            return (cost, idx, k)
        return None

    while True:
        rc = candidate(rh, self.rows, rversions, self.cols)
        cc = candidate(ch, self.cols, cversions, self.rows)
        if rc is None and cc is None:
            break
        if cc is None or (rc is not None and rc[0] <= cc[0]):
            _, r, c = rc
        else:
            _, c, r = cc
        a = self.rows[r][c]
        assert abs(a) == 1
        prow = list(self.rows[r].items())
        pcol = list(self.cols[c].items())
        touched_rows = {i for i, _ in pcol}
        touched_cols = {j for j, _ in prow}
        if on_pivot is not None:
            on_pivot(r, c, a)
        for j, b in prow:
            if j == c:
                continue
            factor = a * b
            col = self.cols[j]
            del col[r]
            for i, x in pcol:
                if i == r:
                    continue
                val = col.get(i, 0) - factor * x
                if val:
                    col[i] = val
                    self.rows[i][j] = val
            else:
                col.pop(i, None)
                self.rows[i].pop(j, None)

```

```

        for i, _ in pcol:
            self.rows[i].pop(c, None)
        self.rows[r] = {}
        self.cols[c] = {}
        for i in touched_rows:
            rversions[i] += 1
            if self.rows[i]:
                heapq.heappush(rh, (len(self.rows[i]), i, rversions[i]))
        for j in touched_cols:
            cversions[j] += 1
            if self.cols[j]:
                heapq.heappush(ch, (len(self.cols[j]), j, cversions[j]))
        rank += 1
        if len(rh) > max(10000, 4 * len(self.rows)):
            rh = [
                (len(row), i, rversions[i])
                for i, row in enumerate(self.rows)
                if row
            ]
            heapq.heapify(rh)
        if len(ch) > max(10000, 4 * len(self.cols)):
            ch = [
                (len(col), j, cversions[j])
                for j, col in enumerate(self.cols)
                if col
            ]
            heapq.heapify(ch)
    return rank

def core_edges():
    return (
        [(0, 1)] * 3
        + [(0, 2), (0, 3), (1, 4), (1, 5)]
        + list(itertools.combinations(range(2, 6), 2))
    )

class EquivariantAbrams:

    def __init__(self):
        n = 4
        core = core_edges()
        edges = []
        for e, (u, v) in enumerate(core):
            x, y = (6 + 2 * e, 7 + 2 * e)
            edges.extend([(u, x), (x, y), (y, v)])
        self.graph = SimpleNamespace(vertices=32, edges=edges)
        self.n = n
        self.group = SymmetricGroup(n)
        self.cells = []
        self.lookup = []
        vertices = self.graph.vertices
        for q in range(n + 1):
            cells = []
            for edgeids in itertools.combinations(range(len(self.graph.edges)), q):
                occupied = set()
                valid = True
                for edge in edgeids:
                    endpoints = set(self.graph.edges[edge])
                    if occupied & endpoints:
                        valid = False
                        break

```

```

        occupied.update(endpoints)
    if not valid:
        continue
    free = [v for v in range(vertices) if v not in occupied]
    for stationary in itertools.combinations(free, n - q):
        cells.append(stationary + tuple((vertices + e for e in edgeids)))
    cells.sort()
    self.cells.append(cells)
    self.lookup.append({cell: i for i, cell in enumerate(cells)})
self.boundaries = {}

def build_boundaries(self):
    vertices = self.graph.vertices
    for q in [3]:
        columns = []
        for cell in self.cells[q]:
            col = {}
            for k, token in enumerate(cell):
                if token < vertices:
                    continue
                edge_position = k - (self.n - q)
                for endpoint, factor in zip(
                    self.graph.edges[token - vertices], (-1, 1)
                ):
                    face = list(cell)
                    face[k] = endpoint
                    permutation = tuple(
                        sorted(range(self.n), key=lambda i: face[i])
                    )
                    target = tuple((face[i] for i in permutation))
                    target_index = self.lookup[q - 1][target]
                    g = self.group.index[permutation]
                    value = factor * (-1) ** edge_position
                    entry = col.setdefault(target_index, {})
                    entry[g] = entry.get(g, 0) + value
                    if not entry[g]:
                        del entry[g]
                    if not entry:
                        del col[target_index]
            columns.append(col)
        self.boundaries[q] = GroupRingMatrix(
            self.group, len(self.cells[q - 1]), columns
        )

def bezout_vector(values):
    """Return g>=0 and coefficients with sum(values[i]*coefficients[i])=g."""
    coefficients = {}
    gcd = 0
    for key, value in values.items():
        if not value:
            continue
        old_r, r = (gcd, value)
        old_s, s = (1, 0)
        old_t, t = (0, 1)
        while r:
            quotient = old_r // r
            old_r, r = (r, old_r - quotient * r)
            old_s, s = (s, old_s - quotient * s)
            old_t, t = (t, old_t - quotient * t)
        if old_r < 0:
            old_r, old_s, old_t = (-old_r, -old_s, -old_t)
        coefficients = {i: a * old_s for i, a in coefficients.items() if a * old_s}

```

```

        if old_t:
            coefficients[key] = coefficients.get(key, 0) + old_t
        gcd = old_r
        if gcd == 1:
            break
    assert sum((values[i] * a for i, a in coefficients.items())) == gcd
    return (gcd, coefficients)

def integer_certificate(matrix):
    records = []

    def pivot(r, c, a):
        records.append(
            (
                r,
                c,
                a,
                tuple(((j, b) for j, b in matrix.rows[r].items() if j != c)),
                tuple(((i, x) for i, x in matrix.cols[c].items() if i != r)),
            )
        )

    pivots = matrix.unit_reduce(on_pivot=pivot)
    nonzero = {j: col for j, col in enumerate(matrix.cols) if col}
    assert nonzero
    first = next(iter(nonzero.values()))
    divisor = math.gcd(*first.values())
    z = {i: a // divisor for i, a in first.items()}
    gcd, functional = bezout_vector(z)
    assert gcd == 1
    scalars = {}
    for j, col in nonzero.items():
        scalar = sum((a * col.get(i, 0) for i, a in functional.items()))
        assert col == {i: scalar * a for i, a in z.items() if scalar * a}
        scalars[j] = scalar
    order, source = bezout_vector(scalars)
    assert order == 6
    phi = {i: a % order for i, a in functional.items() if a % order}
    for r, c, a, prow, pcol in reversed(records):
        coefficient = -a * sum((b * source.get(j, 0) for j, b in prow))
        if coefficient:
            source[c] = coefficient
        value = -a * sum((x * phi.get(i, 0) for i, x in pcol)) % order
        if value:
            phi[r] = value
    return (z, source, phi, pivots)

def construct():
    model = EquivariantAbrams()
    model.build_boundaries()
    matrix = model.boundaries[3]
    records = []

    def pivot(r, c, unit):
        g, sign = next(iter(unit.items()))
        records.append(
            (
                r,
                c,
                g,
                sign,
            )
        )

```

```

        tuple((j, a) for j, a in matrix.rows[r].items() if j != c)),
        tuple((i, a) for i, a in matrix.cols[c].items() if i != r)),
    )
)

gpivots = matrix.unit_reduce(on_pivot=pivot)
rowids = [i for i, row in enumerate(matrix.rows) if row]
colids = [j for j, col in enumerate(matrix.cols) if col]
assert gpivots == 131107
assert (len(rowids), len(colids)) == (3361, 61)
rindex = {r: i for i, r in enumerate(rowids)}
group = model.group
N = len(group.permutations)
multiplication = [[group.multiply(g, h) for h in range(N)] for g in range(N)]
right_shifts = [[multiplication[g][h] for g in range(N)] for h in range(N)]
columns = []
for c in colids:
    for g in range(N):
        col = {}
        for r, a in matrix.cols[c].items():
            offset = rindex[r] * N
            for h, value in a.items():
                col[offset + multiplication[g][h]] = value
        columns.append(col)
integer = SparseMatrix(len(rowids) * N, columns)
z, source, phi, ipivots = integer_certificate(integer)
assert ipivots == 118
del integer
zgroup = {}
for i, a in z.items():
    r, g = divmod(i, N)
    zgroup.setdefault(rowids[r], {})[g] = a
bgroup = {}
for j, a in source.items():
    c, g = divmod(j, N)
    bgroup.setdefault(colids[c], {})[g] = a
phigroup = {}
for i, a in phi.items():
    r, g = divmod(i, N)
    phigroup.setdefault(rowids[r], [0] * N)[g] = a
for r, c, g, sign, prow, pcol in reversed(records):
    inverse = group.inverses[g]
    coefficient = {}
    for j, a in prow:
        value = bgroup.get(j)
        if value:
            group.add_product(coefficient, value, a)
    if coefficient:
        bgroup[c] = {
            multiplication[h][inverse]: -sign * a for h, a in coefficient.items()
        }
values = [0] * N
for i, a in pcol:
    functional = phigroup.get(i)
    if functional is None:
        continue
    for h, coefficient in a.items():
        factor = -sign * coefficient
        if factor % 6 == 0:
            continue
        shift = right_shifts[multiplication[inverse][h]]
        for u in range(N):
            values[u] += factor * functional[shift[u]]

```

```

        values = [x % 6 for x in values]
        if any(values):
            phigroup[r] = values
del records
result = {
    "modulus": 6,
    "particle_count": 4,
    "core_graph": {
        "vertices": 6,
        "oriented_edges": [list(e) for e in core_edges()],
    },
    "subdivided_graph": {
        "vertices": model.graph.vertices,
        "oriented_edges": model.graph.edges,
    },
    "permutations": [list(p) for p in group.permutations],
    "unlabelled_cell_counts": list(map(len, model.cells)),
    "cycle_z": [
        [list(model.cells[2][r]), sorted(a.items())]
        for r, a in sorted(zgroup.items())
    ],
    "bounding_chain_b": [
        [list(model.cells[3][c]), sorted(a.items())]
        for c, a in sorted(bgroup.items())
    ],
    "detecting_cocycle_phi": [
        [list(model.cells[2][r]), a] for r, a in sorted(phigroup.items())
    ],
    "construction": {
        "group_ring_unit_pivots": gpivots,
        "integer_unit_pivots": ipivots,
    },
}
return result

def check(certificate):
    if not __debug__:
        raise RuntimeError(
            "Run this verifier without -O; its assertions are the certificate checks."
        )
    assert certificate["particle_count"] == 4
    assert certificate["modulus"] == 6
    core = (
        [(0, 1)] * 3
        + [(0, 2), (0, 3), (1, 4), (1, 5)]
        + list(itertools.combinations(range(2, 6), 2))
    )
    assert certificate["core_graph"] == {
        "vertices": 6,
        "oriented_edges": [list(e) for e in core],
    }
    expected_edges = []
    for e, (u, v) in enumerate(core):
        x, y = (6 + 2 * e, 7 + 2 * e)
        expected_edges.extend([(u, x), (x, y), (y, v)])
    graph = certificate["subdivided_graph"]
    vertices = graph["vertices"]
    edges = [tuple(e) for e in graph["oriented_edges"]]
    assert vertices == 32 and edges == expected_edges and (len(edges) == 39)
    adjacency = [[] for _ in range(vertices)]
    for e, (u, v) in enumerate(edges):
        adjacency[u].append((v, e))

```

```

adjacency[v].append((u, e))

def distances(start, omitted_edge=None):
    distance = {start: 0}
    queue = deque([start])
    while queue:
        u = queue.popleft()
        for v, e in adjacency[u]:
            if e != omitted_edge and v not in distance:
                distance[v] = distance[u] + 1
                queue.append(v)
    return distance

assert len(distances(0)) == vertices
special = [v for v in range(vertices) if len(adjacency[v]) != 2]
assert special == list(range(6))
shortest_special_path = min(
    (distances(u)[v] for u, v in itertools.combinations(special, 2))
)
girth = min((distances(u, e).get(v, 10**9) + 1 for e, (u, v) in enumerate(edges)))
assert shortest_special_path >= 3 and girth >= 5
permutations = list(itertools.permutations(range(4)))
assert certificate["permutations"] == [list(p) for p in permutations]
pindex = {p: i for i, p in enumerate(permutations)}
multiply = [
    [pindex[tuple((p[q[i]] for i in range(4)))] for q in permutations]
    for p in permutations
]
zero = (0,) * 24

def validate_cell(cell, dimension):
    assert len(cell) == 4 and all((type(x) is int for x in cell))
    assert tuple(sorted(cell)) == cell and len(set(cell)) == 4
    assert sum((x >= vertices for x in cell)) == dimension
    occupied = 0
    for token in cell:
        assert 0 <= token < vertices + len(edges)
        if token < vertices:
            closure = 1 << token
        else:
            u, v = edges[token - vertices]
            closure = 1 << u | 1 << v
        assert not occupied & closure, ("intersecting cell closures", cell)
        occupied |= closure

def read_chain(name, dimension):
    chain = {}
    for cell, entries in certificate[name]:
        cell = tuple(cell)
        validate_cell(cell, dimension)
        assert cell not in chain
        values = [0] * 24
        for p, a in entries:
            assert type(p) is int and 0 <= p < 24
            assert type(a) is int and a != 0 and (values[p] == 0)
            values[p] = a
        assert any(values)
        chain[cell] = values
    return chain

z = read_chain("cycle_z", 2)
assert len(z) == 409
signs = [

```

```

        (-1) ** sum(p[i] > p[j] for i in range(4) for j in range(i + 1, 4))
        for p in permutations
    ]
    assert all(
        values[0] in (-1, 1) and values == [values[0] * a for a in signs]
        for values in z.values()
    )
    b = read_chain("bounding_chain_b", 3)
    phi = {}
    for cell, values in certificate["detecting_cocycle_phi"]:
        cell = tuple(cell)
        validate_cell(cell, 2)
        assert cell not in phi and len(values) == 24
        assert all((type(x) is int and 0 <= x < 6 for x in values)) and any(values)
        phi[cell] = values

    def faces(cell):
        """Boundary of the canonical oriented cube, with its label maps."""
        axis = 0
        for coordinate, token in enumerate(cell):
            if token < vertices:
                continue
            for endpoint, end_sign in zip(edges[token - vertices], (-1, 1)):
                ordered_face = list(cell)
                ordered_face[coordinate] = endpoint
                old_coordinates = tuple(
                    sorted(range(4), key=lambda i: ordered_face[i])
                )
                canonical_face = tuple((ordered_face[i] for i in old_coordinates))
                yield (
                    canonical_face,
                    pindex[old_coordinates],
                    end_sign * (-1) ** axis,
                )
            axis += 1

    def boundary(chain):
        answer = {}
        for cell, coefficients in chain.items():
            support = [(p, a) for p, a in enumerate(coefficients) if a]
            for face, q, sign in faces(cell):
                row = answer.setdefault(face, [0] * 24)
                for p, a in support:
                    row[multiply[p][q]] += sign * a
            return {cell: values for cell, values in answer.items() if any(values)}

    assert not boundary(z), "The proposed 2-chain is not a cycle."
    expected_boundary = {cell: [6 * a for a in values] for cell, values in z.items()}
    assert (
        boundary(b) == expected_boundary
    ), "The displayed 3-chain does not bound 6*z."
    evaluation = (
        sum(
            (
                a * phi.get(cell, zero)[p]
                for cell, values in z.items()
                for p, a in enumerate(values)
            )
        )
        % 6
    )
    assert evaluation == 1, (
        "The cocycle does not detect z with value 1.",
    )

```

```

        evaluation,
    )
    unlabelled_cubes = 0
    for edge_ids in itertools.combinations(range(len(edges)), 3):
        endpoints = [v for e in edge_ids for v in edges[e]]
        if len(set(endpoints)) != 6:
            continue
        occupied = set(endpoints)
        edge_tokens = tuple((vertices + e for e in edge_ids))
        for stationary in range(vertices):
            if stationary in occupied:
                continue
            cell = (stationary,) + edge_tokens
            evaluations = [0] * 24
            for face, q, sign in faces(cell):
                values = phi.get(face)
                if values is None:
                    continue
                for p in range(24):
                    evaluations[p] += sign * values[multiply[p][q]]
            assert all((value % 6 == 0 for value in evaluations)), (
                "Cocycle condition fails at",
                cell,
                evaluations,
            )
            unlabelled_cubes += 1
    assert unlabelled_cubes == 174772
    assert certificate["unlabelled_cell_counts"][3] == unlabelled_cubes
    return {
        "verified": True,
        "conclusion": "[z] has exact order 6 in H2(F4(G); Z).",
        "subdivided_graph_vertices": vertices,
        "subdivided_graph_edges": len(edges),
        "shortest_path_between_special_vertices": shortest_special_path,
        "girth": girth,
        "labelled_3_cubes_checked": 24 * unlabelled_cubes,
        "cycle_nonzero_coefficients": sum(
            (sum((bool(a) for a in values)) for values in z.values())
        ),
        "bounding_chain_nonzero_coefficients": sum(
            (sum((bool(a) for a in values)) for values in b.values())
        ),
        "cocycle_nonzero_values": sum(
            (sum((bool(a) for a in values)) for values in phi.values())
        ),
        "cocycle_evaluation_on_z_mod_6": evaluation,
    }

if __name__ == "__main__":
    if not __debug__:
        raise RuntimeError("Run without -O: assertions must be enabled.")
    certificate = construct()
    result = check(certificate)
    assert result["cycle_nonzero_coefficients"] == 9816
    assert result["bounding_chain_nonzero_coefficients"] == 236992
    assert result["cocycle_nonzero_values"] == 214457
    result["construction"] = certificate["construction"]
    print(json.dumps(result, indent=2))

```